

Number: P3125R3
Date: 2025-01-13
Audience: LEWG
Reply-to: [Hana.Dusíková](#)

Table of contents

- [Revision history](#)
- [Introduction and motivation](#)
 - [Use cases](#)
 - [Safety](#)
 - [Unsafe operations](#)
 - [Examples](#)
 - [HAMT early leaves](#)
 - [Smart \(non-\)owning pointer](#)
 - [LLVM's L-value](#)
- [Implementation experience](#)
 - [Implementation in the library](#)
 - [Accessing raw tagged pointers](#)
 - [Implementation in the compiler](#)
 - [Compiler builtins](#)
 - [Constant evaluation](#)
 - [Pointer provenance and optimization](#)
 - [Alternative constexpr compatible implementation](#)
- [Design](#)
 - [Preconditions and eligibility of constructors](#)
 - [Representation of tag value](#)
 - [Converting to bit representation](#)
 - [Tuple protocol](#)
- [Things it's not doing and why](#)
- [Impact on existing code](#)
- [Proposed changes to wording](#)
 - [Feature test macro](#)

constexpr pointer tagging

This paper proposes a new library non-owning pointer and value pair which has ability to store small amount of information in a *tag* in alignment low-bits. This functionality is also usable in `constexpr` environment so. It's meant to be a building tool to build more advanced data-structures and also last requirement for `atomic<std::shared_ptr<T>>` to be `constexpr`.

Revision history

- [R2](#) → [R3](#): polishing design and [added wording](#)
- [R1](#) → [R2](#): simplifying design by removing schemas and the heavy interface, massive simplification as discussed in SG1 (`pointer_tag_pair`, required `sizeof(pointer_tag_pair) == sizeof(T*)`)
- [R0](#) → [R1](#): proposing and explaining design based on existing implementation

Introduction and motivation

Pointer tagging is widely known and used technique ([Glasgow Haskell Compiler](#), LLVM's `PointerIntPair`, `PointerUnion`, CPython's garbage collector, `Objective-C` / Swift, Chrome's [V8 JavaScript engine](#), `GAP`, `OCaml`, `PBRT`). All major CPU vendors provides mechanism for pointer tagging (Intel's LAM [linear address masking](#), AMD's [Upper Address Ignore](#), ARM's TBI [top byte ignore](#) and MTE [memory tagging extension](#)). All widely used 64 bit platforms are [not using more than 48 or 49 bits of the pointers](#).

This functionality widely supported can't be expressed in a standard conforming way.

[Rust](#), [Dlang](#), or [Zig](#) has an interface for